paula na frente
UNIOESTE

Ana Abati, Eduarda Cunha, Paula Korber

12 de setembro de 2025

Índice

1 Grafos **2**

1.1 Biconnected Components . . . 2
1.2 BlockCut Tree . . . 3
1.3 Bridge Tree . . . 3
1.4 Bridge Tree . . . 3
1.5 Dijkstra . . . 3
1.6 Disjoint Set Union . . . 4
1.7 Euler Tour . . . 4
1.8 Floyd-Warshall . . . 4
1.9 Kahns Algorithm . . . 5
1.10 Kosaraju . . . 5
1.11 LCA . . . 6
1.12 LCA with RMQ . . . 6
1.13 Pontos de Articulação . . . 6
1.14 Tree Center . . . 7
1.15 Tree Distances . . . 8

2 Estruturas **8**

2.1 2D Prefix Sum . . . 8
2.2 Binary Indexed Tree/Fenwick Tree . . . 8
2.3 Binary Search . . . 9
2.4 Lazy SegTree . . . 9
2.5 SegTree . . . 10

3 DP **10**

3.1 Coins Combinations I . . . 10
3.2 Coins Combinations II . . . 11
3.3 Edit Distance . . . 11
3.4 Kadane . . . 11
3.5 Knapsack . . . 11
3.6 Longest Common Subsequence . . . 11
3.7 Longest Increasing Subsequence . . . 12
3.8 Minimizing Coins . . . 12
3.9 Small To Large example . . . 13

4 Miscellaneous **13**

4.1 Custom Comparator 13

5 String **13**

5.1 ing int64 = long long; 13

5.2 Manacher 13

6 Matematica **14**

6.1 FastExpo 14

6.2 Fatorial e Inverso Modular 14

6.3 GCD 14

6.4 Prime Factorization 14

6.5 Smallest Prime Factor 15

7 Problemas **15**

7.1 Array Division 15

7.2 Distinct Value Queries 15

7.3 Divisor Analysis 16

7.4 Increasing Subsequence II 16

7.5 Sum of Divisors 17

8 Extra **18**

8.1 bin.cpp 18

8.2 template.cpp 19

8.3 makefile 19

1 Grafos

1.1 Biconnected Components

```
009 void novaComponenteBiconexa(int idAresta){
6a4     c++;
187     int a;
016     do {
f55         a = pilhaArestas[p];
506         p--;
92a         componenteDaAresta[a] = c;
61e         if(p == -1) break;
61f     }while(a != idAresta);
6f0 }

955 void dfs(int v, int p){
cda     visited[v] = 1;
98b     low[v] = pre[v] = ++t;
938     int qtdFilhos = 0;
284     for (auto u : graph[v]){
022         int viz = graph[v][i];
cf2         int idAresta = aresta[v][i];
d7b         if(visitedAresta[idAresta] == 1) continue;
e4f         visitedAresta[idAresta] = 1;
fb2         p++;
922         pilhaAresta[p] = idAresta;
264         if (!visited[u]){
412             dfs(u,v);
c80             low[v] = min(low[v], low[u]);

565             bool achouNovaComponente = false;
33c             if(v == 1 && qtdFilhos >= 2) achouNovaComponente =
    false;
0ed             if(v != 1 && low[u] >= low[v]) achouNovaComponente =
    true;

68b             if(achouNovaComponente){
20e                 pontoDeArticulacao[v] = 1;
0ec                 novaComponenteBiconexa(idAresta);
bc0             }
18d         }
4e6         else {
39b             low[v] = min(low[v], pre[u]);
0f7         }
5a6     }
8d5 }


//t = 0, p = -1, c = 0;
//dfs(0, 0);
```

```
//novaComponenteBiconexa(-1);
```

1.2 BlockCut Tree

```
78a for(int i = 1; i <= n; i++){
acf     if(pontoDeArticualacao[i] != 1) continue;

6a4     c++;
620     pontoDeArticulacao[i] = c;
3a2     for(int j = 0; j < aresta[i].size(); j++){
4e7         int idAresta = aresta[i][j];
db7         int comp = componenteDaAresta[idAresta];
2f5         if(marcComp[comp] == 0){
11a             marcComp[comp] = 1;
886             blockCutTree[pontoDeArticulacao[i]].push_back(comp);
070             blockCutTree[comp].push_back(pontoDeArticulacao[i]);
daa         }
8b7     }

3a2     for(int j = 0; j < aresta[i].size(); j++){
4e7         int idAresta = aresta[i][j];
db7         int comp = componenteDaAresta[idAresta];
fcd         marcComp[comp] = 0;
963     }
c4b }
```

1.3 Bridge Tree

```
9fa int t, c, low[MAX], pre[MAX], visited[MAX], comp[MAX];
7b4 stack<int> st;
375 vector<int> graph[MAX], compgraph[MAX];

955 void dfs(int v, int p){
cda     visited[v] = 1;
98b     low[v] = pre[v] = ++t;
30c     st.emplace(v);
284     for (auto u : graph[v]){
264         if (!visited[u]){
412             dfs(u,v);
c80             low[v] = min(low[v], low[u]);
77d         }
4e6         else {
2da             if (u == p) continue;
39b             low[v] = min(low[v], pre[u]);
acd         }
11d     }
cf1     if (low[v] == pre[v]){ //achamos uma bridge
6a4         c++;
2f4         int cur;
016         do {
596             cur = st.top();
25a             st.pop();
524             comp[cur] = c;
cca             compgraph[c].push_back(cur);
583         } while(cur != v);
b04     }
6fc }
```

1.4 Bridge Tree

```
9fa int t, c, low[MAX], pre[MAX], visited[MAX], comp[MAX];
7b4 stack<int> st;
375 vector<int> graph[MAX], compgraph[MAX];

955 void dfs(int v, int p){
cda     visited[v] = 1;
98b     low[v] = pre[v] = ++t;
30c     st.emplace(v);
284     for (auto u : graph[v]){
264         if (!visited[u]){
412             dfs(u,v);
c80             low[v] = min(low[v], low[u]);
77d         }
4e6         else {
2da             if (u == p) continue;
39b             low[v] = min(low[v], pre[u]);
acd         }
11d     }
        //aq depende se ta 0 ou 1 indexado
6ee     if (v != 0 && low[v] == pre[v]) resp.push_back({v, pai});
48b }
```

1.5 Dijkstra

```
5c1 const int MAXN = 2e5+7;
222 const int INF = 1e18;
c31 vector<pair<int, int>> graph[MAXN];
```

```
9d3 int dist[MAXN];
14e int n, m;

922 void dijkstra(){
        //dist, node
fe1     priority_queue<pair<int, int>> pq;
3be     pq.push({0, 0}); //0 indexed
ca1     for(int i = 0; i < MAXN; i++) dist[i] = INF;
50a     dist[0] = 0;
502     while(!pq.empty()){
174         auto [d, v] = pq.top(); pq.pop();
842         d = -d;
68b         if(dist[v] < d) continue;
940         for(auto [u, w] : graph[v]){
911             if(dist[u] > dist[v] + w){
608                 dist[u] = dist[v] + w;
4f1                 pq.push({-dist[u], u});
1ec             }
1a7         }
7bb     }
ff2     for(int i = 0; i < n; i++) cout << dist[i] << " ";
1fb     cout << endl;
e1e }
```

1.6 Disjoint Set Union

```
//implements rank heuristic + path compression
d56 struct DSU{
1a8     int n;
632     vector<int> p;
02d     vector<int> r;
f4e     DSU(int _n){
8fd         n = _n;
1c4         p.resize(n);
7e4         r.resize(n);
603         for(int i = 0; i < n; i++){
f13             p[i] = i;
fd1             r[i] = 0;
606         }
26a     }

57b     int find(int x){
eab         return (p[x] == x? x : p[x] = find(p[x]));
ab9     }

440     void unite(int a, int b){
605         a = find(a), b = find(b);
d54         if(a == b) return;
dac         if(r[a] == r[b]) r[a]++;
d1b         if(r[a] > r[b]){
67e             r[a] += r[b];
264             p[b] = a;
dfa         }else{
2e3             r[b] += r[a];
84b             p[a] = b;
308         }
07e     }

671 };
```

1.7 Euler Tour

```
642 const int MAXN = 2*(1e5)+7;
47b vector<int> graph[MAXN];
36f int tin[MAXN], tout[MAXN], visited[MAXN];
2dc int timer = 0;
76a void dfs(int v){
237     tin[v] = ++timer;
cda     visited[v] = 1;
284     for(auto u : graph[v]){
e39         if(!visited[u]) dfs(u);
6e1     }
a66     tout[v] = timer;
166 }
```

1.8 Floyd-Warshall

```
//O(N^3)

c4a const int MAXN = 503;
7bf int graph[MAXN][MAXN];
299 int dist[MAXN][MAXN];

10c const int INF = 1e13;
421 void floydwarshal(){
14d     int n, m, q; cin >> n >> m >> q;
        //leitura do grafo
78a     for(int i = 1; i<= n; i++){
160         for(int j = 1; j<= n; j++){
57a             if(i == j) dist[i][j] = 0;
```

```
d65             else dist[i][j] = INF;
d9a         }
dca     }
c29     for(int i = 1; i <= m; i++){
32e         int a, b, c; cin >> a >> b >> c;
73f         if(dist[a][b] > c) dist[a][b] = dist[b][a] = c;
fdb     }

5d3     for(int k = 1; k <= n; k++){
78a         for(int i = 1; i <= n; i++){
160             for(int j = 1; j <= n; j++){
4c4                 dist[i][j] = min(dist[i][j], dist[i][k] +
    dist[k][j]);
946             }
44b         }
283     }
e36 }
```

1.9 Kahns Algorithm

```
//Remember to fill indeg array and check MAXN
245 const int MAXN = 2*1e5+8;
47b vector<int> graph[MAXN];
759 int indeg[MAXN];
9d1 void topo(){
1a8     int n;
797     queue<int> pq;
603     for(int i = 0; i < n; i++){
016         if(!indeg[i]) pq.push(i);
b91     }

48b     vector<int> toposort;
502     while(!pq.empty()){
a7f         auto v = pq.front(); pq.pop();
c50         toposort.push_back(v);
284         for(auto u : graph[v]){
4e9             if(--indeg[u] == 0) pq.push(u);
8dc         }
d2d     }
187 }
```

1.10 Kosaraju

```
e34 vector<int> graph[MAXN], reverso[MAXN];
ee3 vector<int> ordemSaida;
e2d int visited[MAXN], comp[MAXN];

76a void dfs(int v){
cda     visited[v] = 1;
284     for(auto u : graph[v]){
e39         if(!visited[u]) dfs(u);
6e1     }
c92     ordemSaida.push_back(v);
ec9 }

58f void dfs2(int v, int c){
eec     visited[v] = 2;
d5f     comp[v] = c;
bb3     for(auto u : reverso[v]){
7e7         if(visited[u] == 1) dfs2(u, c);
a31     }
8f6 }
e8d int main(){
acf     int n, m; cin >> n >> m;

c29     for(int i = 1; i <= m; i++){
b1f         int a, b; cin >> a >> b;
8fa         graph[a].push_back(b);
9e9         reverso[b].push_back(a);
205     }
78a     for(int i = 1; i <= n; i++){
312         if(!visited[i]) dfs(i);
c9b     }

aff     int c = 0;
b19     for(int i = ordemSaida.size() - 1; i >= 0; i--){
38c         if(visited[ordemSaida[i]] == 1){
6a4             c++;
6e8             dfs2(ordemSaida[i], c);
83c         }
554     }

262     cout << c << endl;
78a     for(int i = 1; i <= n; i++){
411         cout << comp[i] << " ";
99b     }
1fb     cout << endl;
ae0 }
```

1.11 LCA

```
bc7 const int MAXL = 20;
dc1 const int MAXN = 2 * (int)(1e5) + 7;
47b vector<int> graph[MAXN];
fd9 int depth[MAXN], anc[MAXN][MAXL];

955 void dfs(int v, int p) {
e1e     anc[v][0] = p;
12b     depth[v] = depth[p] + 1;
f3a     for (int i = 1; i < MAXL; i++) {
f5d         anc[v][i] = anc[ anc[v][i-1] ][i-1];
fce     }
284     for (auto u : graph[v]) {
58d         if (u != p) {
412             dfs(u, v);
bb2         }
ab6     }
70b }

7be int lca(int a, int b){
a0c     if(depth[a] < depth[b]) swap(a, b);
caa     int k = depth[a] - depth[b];
5b7     for(int i = MAXL-1; i >= 0; i--){
dd4         if(k&(1<<i)){
13a             a = anc[a][i];
e01         }
66b     }

6df     if(a == b) return a;

5b7     for(int i = MAXL-1; i >= 0; i--){
358         if(anc[a][i] != anc[b][i]){
13a             a = anc[a][i];
c8d             b = anc[b][i];
c81         }
627     }
ef4     return anc[a][0];
d67 }
```

1.12 LCA with RMQ

```
//query: O(1)
3a9 const int LOG = 30;
b40 const int MAX = 1e5+7;
4f6 int n, timer = 0, tin[MAX], depth[MAX];
ba0 vector<int> et;
227 pair<int,int> sp[MAX*2][LOG+1];
b24 vector<int> graph[MAX];

955 void dfs(int v, int p){

8c5     tin[v] = et.size();
490     et.push_back(v);
284     for (auto u : graph[v]){
58d         if (u!=p){
cf4             depth[u] = depth[v]+1;
412             dfs(u,v);
490             et.push_back(v);
f82         }
7ac     }
6e0 }

07b void buildtable(){
96d     for (int i = 0; i < (int)et.size(); i++){
1e7         sp[i][0] = {depth[et[i]], et[i]};
bcc     }
2ae     for (int j = 1; j <= LOG; j++){
538         for (int i = 0; i + (1<<(j-1)) < (int)et.size(); i++){
d30             sp[i][j] = min(sp[i][j-1], sp[i+(1<<(j-1))][j-1]);
985         }
fd8     }
641 }

2b4 pair<int,int> query (int a, int b){
bfb     int len =  b - a + 1;
5e4     int lg = 31 - __builtin_clz(len);
a57     return min(sp[a][lg], sp[b - (1<<lg) + 1][lg]);
11c }

7be int lca(int a, int b){
fb2     if (tin[a] > tin[b]) swap(a,b);
ce1     return query(tin[a], tin[b]).second;
940 }
```

1.13 Pontos de Articulação

```
955 void dfs(int v, int p){
cda     visited[v] = 1;
98b     low[v] = pre[v] = ++t;
```

```
14e     bool art = false;
236     int filhos = 0;
284     for (auto u : graph[v]){
264         if (!visited[u]){
701             filhos++;
412             dfs(u,v);
c80             low[v] = min(low[v], low[u]);

b71             if(low[v] >= pre[v]) art = true;
4d8         }
4e6         else {
2da             if (u == p) continue;
39b             low[v] = min(low[v], pre[u]);
acd         }
f4f     }
ca1     if (v == 1 && filhos >= 2) ans.push_back(v);
021     if (v != 1 && art) ans.push_back(v);
7fd }
```

1.14 Tree Center

```
1a8 int n;
69f void dfs(int v, int p, int id){
284     for(auto u : graph[v]){
2ab         if(u == p || dist[id][u] > 0) continue;
14a         dist[id][u] = dist[id][v] + 1;
041         dfs(u, v, id);
0ea     }
d8f }

1a3 int achador(int id){
152     int mx = 0, u = -1;
603     for(int i = 0; i < n; i++){
efa         if(dist[id][i] > mx){
87e             mx = dist[id][i];
af1             u = i;
cce         }
87a     }
03f     return u;
865 }

a94 int dfs2(int v, int p, int d){
e40     if(d == 0) return 1;
1a4     int ans = 0;
284     for(auto u : graph[v]){
58d         if(u != p){
591             ans += dfs2(u, v, d-1);
cd8         }
241     }
ba7     return ans;
0e0 }

e8d int main(){
a68     cin >> n;
f50     for(int i = 0; i < n-1; i++){
b1f         int a, b; cin >> a >> b;
8fa         graph[a].push_back(b);
4c6         graph[b].push_back(a);
d60     }

c7e     dfs(1, 0, 0);
02e     int u = achador(0);

       // cout << u << endl;

f75     dfs(u, 0, 1);
44f     int v = achador(1);
        //cout << v << endl;
2ef     dfs(v, 0, 2);
        //cout << dist[1][v] << endl;

6c9     vector<int> centro;
78a     for(int i = 1; i <= n; i++){
            //cout << i << " " << dist[1][i] << " " << dist[2][i] <<
                endl;
7b4         if(dist[1][i] + dist[2][i] == dist[1][v] && abs(dist[1][i]
    - dist[2][i]) <= 1){
d6f             centro.push_back(i);
505         }
393     }
37d     if(centro.size() == 1){
699         cout << dist[1][v]+1 << endl;
6de         int ans = 0, sum = 0;
d3d         for(auto u : graph[centro[0]]){
                //cout << "centro " << dist[1][v]/2 << endl;
4d7             int qtdf = dfs2(u, centro[0], dist[1][v]/2-1);
                //cout << qtdf << endl;
a26             sum += qtdf;
0c4             ans -= (qtdf*qtdf - qtdf)/2;
d50         }
bef         ans += (sum*sum - sum)/2;
886         cout << ans << endl;
fb3     }else{
```

```
699         cout << dist[1][v]+1<< endl;
c96         int qtd1 = dfs2(centro[0], centro[1], dist[1][v]/2);
a5e         int qtd2 = dfs2(centro[1], centro[0], dist[1][v]/2);
d5e         cout << qtd1*qtd2 << endl;
4c7     }
01a }
```

1.15 Tree Distances

```
955 void dfs(int v, int p){
284     for(auto u : graph[v]){
2da         if(u == p) continue;
412         dfs(u, v);
e77         maxdist[v] = max(maxdist[v], maxdist[u]+1);
53c     }
68e }

be5 void dfs2(int v, int p, int dist){
261     ans[v] = max(maxdist[v], dist);

750     int max1 = 0, max2 = 0;
284     for(auto u : graph[v]){
2da         if(u == p) continue;
020         if(maxdist[u] + 1 > max1){
87e             max2 = max1;
ce9             max1 = maxdist[u]+1;
95f         }else if((maxdist[u]+1) > max2){
003             max2 = maxdist[u]+1;
4e8         }
194     }

284     for(auto u : graph[v]){
2da         if(u == p) continue;
87d         if(maxdist[u]+1 == max1){
524             dfs2(u, v, max(dist, max2)+1);
4a9         }else dfs2(u, v, max(max1, dist)+1);
e70     }
e09 }
```

2 Estruturas

2.1 2D Prefix Sum

```
c5f     int n, q; cin >> n >> q;
78a     for(int i = 1; i <= n; i++){
160         for(int j = 1; j <= n; j++){
602             char a; cin >> a;
52c             if(a == '*')grid[i][j]++;
24e         }
c79     }

78a     for(int i = 1; i <= n; i++){
160         for(int j = 1; j <= n; j++){
be2             grid[i][j] += grid[i-1][j] + grid[i][j-1] -
    grid[i-1][j-1];
1ec         }
d43     }
1f4     while(q--){
d3a         int y1, x1, y2, x2; cin >> y1 >> x1 >> y2 >> x2;
            //pega
e5d         cout << grid[y2][x2] - grid[y1-1][x2] - grid[y2][x1-1] +
    grid[y1-1][x1-1] << endl;
b73     }

cbb }
```

2.2 Binary Indexed Tree/Fenwick Tree

```
//query: O(logN)
714 struct BIT {
1a8     int n;
116     vector<int> bit;
5c7     void init(int _n){
81f         n = _n+1;
c73         bit.resize(n, 0);
267     }

57c     void upd(int x, int v){
aa1         for(; x <= n; x+=x&(-x)) bit[x] += v;
f0d     }

d15     int sum(int x){
37f         int s = 0;
905         for(; x > 0; x -= x&(-x)) s += bit[x];
047         return s;
6a9     }

9e3     int query(int l, int r){
67b         return sum(r) - sum(l-1);
```

```
8d4     }
d2a };
```

2.3 Binary Search

```
cbb }

8bd void binarysearch(){
46c     int l = 1, r = 1e18;
40c     while(l < r){
318         int mid = l+(r-l)/2;
043         if(check(mid)) r = mid;
760         else l = mid+1;
651     }
59a }
```

2.4 Lazy SegTree

```
//rangequery: O(logN)
3de struct lazyseg {
1a8     int n;
7eb     vector<int> t;
c68     vector<int> lazy;
5c7     void init(int _n) {
8fd         n = _n;
dd9         t.assign(4*n, 0);
357         lazy.assign(4*n, 0);
a85     }

123     void propagate(int v, int l, int r){
0f3         if(lazy[v] != 0){
00a             if((r - l + 1) & 1){
5fb                 t[v] ^= lazy[v];
031             }
579             if(l != r){
ace                 lazy[2*v+1] = op(lazy[2*v+1], lazy[v]);
262                 lazy[2*v+2] = op(lazy[2*v+2], lazy[v]);
121             }
51e             lazy[v] = 0;
a41         }
559     }

ab6     int op(int a, int b){
a6d         return (a ^ b);
1b7     }

b89     void updateRange(int v, int l, int r, int lx, int rx, int x){
3fc         propagate(v, l, r);
c86         if(rx < l || r < lx) return;
5c9         if(lx <= l && r <= rx){
035             lazy[v] ^= x;
3fc             propagate(v, l, r);
505             return;
c31         }

ee4         int m = (l+r)/2;
d67         updateRange(2*v+1, l, m, lx, rx, x);
579         updateRange(2*v+2, m+1, r, lx, rx, x);
4ee         t[v] = op(t[2*v+1], t[2*v+2]);
5d6     }

875     void updateRange(int l, int r, int val){
59d         updateRange(0, 0, n-1, l, r, val);
807     }

e6b     int queryRange(int v, int l, int r, int lx, int rx){
3fc         propagate(v, l, r);
f70         if(rx < l || r < lx) return 0;
1ac         if(lx <= l && r <= rx) return t[v];
ee4         int m = (l+r)/2;
382         return op(queryRange(2*v+1, l, m, lx, rx),
    queryRange(2*v+2, m+1, r, lx, rx));
af3     }

9e3     int query(int l, int r){
fd6         return queryRange(0, 0, n-1, l, r);
ec0     }

711     void build(vector<int> &a, int v, int l, int r) {
893         if (l == r) {
28f             t[v] = a[l];
505             return;
167         }
ee4         int m = (l + r)/2;
88c         build(a, 2*v+1, l, m);
717         build(a, 2*v+2, m+1, r);
4ee         t[v] = op(t[2*v+1], t[2*v+2]);
945     }
7f9     void build(vector<int> &a) {
3b5         build(a, 0, 0, n-1);
```

```
86e     }

3bd };
```

2.5 SegTree

```
//query: O(logN)

67a template<typename T>

07c struct segtree{
1a8     int n;
7eb     vector<int> t;
5c7     void init(int _n){
8fd         n = _n;
dd9         t.assign(4*n, 0);
c5f     }

1cb     T op(T a, T b){
23a         return min(a, b);
856     }

476     void build(vector<int> &arr, int v, int l, int r){
893         if(l == r){
d60             t[v] = arr[l];
505             return;
303         }
ee4         int m = (l+r)/2;
ebc         build(arr, 2*v+1, l, m);
618         build(arr, 2*v+2, m+1, r);
4ee         t[v] = op(t[2*v+1], t[2*v+2]);
317     }

6c9     void build(vector<int> &arr){
886         build(arr, 0, 0, n-1);
0dd     }

8fb     int query(int v, int l, int r, int ql, int qr){
dd9         if(qr < l || r < ql) return 0;
639         if(ql <= l && r <= qr) return t[v];
ee4         int m = (l+r)/2;
71d         return op(query(2*v+1, l, m, ql, qr), query(2*v+2, m+1, r,
    ql, qr));
9d9     }

9e3     int query(int l,    int r){
339         if(l > r) return 0;
5d9         return query(0, 0, n-1, l, r);
fbb     }

b22     void update(int v, int l, int r, int idx, int x){
893         if(l == r){
e60             t[v] += x;
505             return;
4c0         }

ee4         int m = (l+r)/2;
5a5         if(idx <= m) update(2*v+1, l, m, idx, x);
19c         else update(2*v+2, m+1, r, idx, x);
4ee         t[v] = op(t[2*v+1], t[2*v+2]);
e80     }

f18     void update(int idx, int x){
d9b         update(0, 0, n-1, idx, x);
19c     }
7b8 };
```

3 DP

3.1 Coins Combinations I

```
0e8     int n, k; cin >> n >> k;
86b     int c[n];
9e4     for(int i = 0; i < n; i++) cin >> c[i];

4c6     vector<int> dp(k+1, 0);
0ec     dp[0] = 1;
db5     for(int i = 1; i <= k; i++){
578         for(int j = 0; j < n; j++){
bca             if(i - c[j] >= 0){
d7b                 dp[i] += dp[i-c[j]];
972                 dp[i] %= MOD;
4a3             }
9df         }
421     }

660     cout << dp[k] << endl;
cbb }
```

3.2 Coins Combinations II

```
0e8     int n, k; cin >> n >> k;
86b     int c[n];
9e4     for(int i = 0; i < n; i++) cin >> c[i];

4c6     vector<int> dp(k+1, 0);
0ec     dp[0] = 1;
603     for(int i = 0; i < n; i++){
acd         for(int j = 1; j <= k; j++){
fec             if(j - c[i]>=0){
d56                 dp[j] += dp[j-c[i]];
3a0                 dp[j] %=MOD;
28a             }
bc5         }
54d     }

660     cout << dp[k] << endl;
cbb }
```

3.3 Edit Distance

```
a77 int dp[MAXN];
574 void editdistance(){
14e     int n, m;
510     for(int i = 0, cnt = 0; i <= max(n, m); i++, cnt++){
85f         dp[i][0] = cnt;
a2b         dp[0][i] = cnt;
f0b     }

        //dp[0][0] = 1;
78a     for(int i = 1; i <= n; i++){
cbc         for(int j = 1; j <= m; j++){
6d5             if(s[i-1] == t[j-1]){
aaf                 dp[i][j] = dp[i-1][j-1];
6a0             }else{
fe7                 int mn = min({dp[i-1][j], dp[i-1][j-1],
    dp[i][j-1]}) + 1;
7cd                 dp[i][j] = mn;
2c3             }
dc1         }
ab2     }
1ad }
```

3.4 Kadane

```
1a8     int n;
a68     cin >> n;
b6d     vector<long long> x(n);
3a8     for (int i = 0; i < n; i++) { cin >> x[i]; }
3ea     ll current_sum = x[0];
de5     ll max_subarray_sum = x[0];
6f5     for (int i = 1; i < n; i++) {
            //Continue the subarray sum or start a new
            //subarray sum beginning at the current element.

eec         current_sum = max(current_sum + x[i], x[i]);
            // Store the maximum subarray sum at each iteration.
31f         max_subarray_sum = max(max_subarray_sum, current_sum);
071     }
706     cout << max_subarray_sum << endl;
cbb }
```

3.5 Knapsack

```
acf     int n, m; cin >> n >> m;
21b     int pages[n], prices[n];
d5e     for(int i = 0; i < n; i++) cin >> prices[i];
36e     for(int i = 0; i < n; i++) cin >> pages[i];

4d4     vector<int> dp(m+1, 0);
603     for(int i = 0; i < n; i++){
775         for(int j = m; j >= prices[i]; j--){
03b             dp[j] = max(dp[j], dp[j-prices[i]] + pages[i]);
2ee         }
231     }

cbb }
```

3.6 Longest Common Subsequence

```
//2. Increase the value of $j$ by $1$ (only works if $j < |B|$).
//3. Increase the value of $i$ and $j$ by $1$ only if $A_i = B_j$.
    Append that
//character $A_i$ (or $B_j$) to the common subsequence. (only works if
//$i < |A|$ and $j < |B|$).
```

```
f9e void lcs(){
a6a     int dp[a.size()][b.size()];
250     for (int i = 0; i < a.size(); i++) { fill(dp[i], dp[i] +
    b.size(), 0); }
bbe     for (int i = 0; i < a.size(); i++) {
273         if (a[i] == b[0]) dp[i][0] = 1;
3b9         if (i != 0) dp[i][0] = max(dp[i][0], dp[i - 1][0]);
8cf     }
b22     for (int i = 0; i < b.size(); i++) {
60d         if (a[0] == b[i]) dp[0][i] = 1;
f5e         if (i != 0) dp[0][i] = max(dp[0][i], dp[0][i - 1]);
d49     }
876     for (int i = 1; i < a.size(); i++) {
cb1         for (int j = 1; j < b.size(); j++) {
b1b             if (a[i] == b[j]) {
008                 dp[i][j] = dp[i - 1][j - 1] + 1;
f36             } else {
398                 dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
f35             }
cf0         }
20d     }
acc     cout << dp[a.size() - 1][b.size() - 1] << endl;

ed8 }
```

3.7 Longest Increasing Subsequence

```
2a0 void lis(){
9ee     int n; cin >> n;
aa8     vector<int> arr(n);
2b5     for(int i = 0; i < n; i++) cin >> arr[i];

471     vector<int> dp;
603     for(int i = 0; i < n; i++){
457         int pos = lower_bound(dp.begin(), dp.end(), arr[i]) -
    dp.begin();
b4d         if(pos == (int) dp.size()){
025             dp.push_back(arr[i]);
f90         }else{
5b7             dp[pos] = arr[i];
aab         }
4b1     }

02e     cout << dp.size() << endl;
0d8 }
```

```
//lis recuperando a sequencia

df5 void lis2(){
9ee     int n; cin >> n;
04a     int arr[n];
2b5     for(int i = 0; i < n; i++) cin >> arr[i];

d04     map<int, int> dp;

1a4     int ans = 0;
936     int lst = 0;
603     for(int i = 0; i < n; i++){
831         dp[arr[i]] = dp[arr[i]-1] + 1;
3ed         if(ans < dp[arr[i]]){
5ce             ans = dp[arr[i]];
e60             lst = arr[i];
caf         }

61a     }

02f     vector<int> res;
45b     for(int i = n-1; i>=0; i--){
4ba         if(arr[i] == lst){
63c             res.push_back(i);
70c             lst--;
8d5         }
742     }

886     cout << ans << endl;
3ea     reverse(res.begin(), res.end());
61a     for(auto u : res){
767         cout << u+1 << " ";
c39     }
02d }
```

3.8 Minimizing Coins

```
0e8     int n, k; cin >> n >> k;
86b     int c[n];
9e4     for(int i = 0; i < n; i++) cin >> c[i];

206     vector<int> dp(k+1, INT_MAX);
076     dp[0] = 0;
603     for(int i = 0; i < n; i++){
296         for(int j = c[i]; j <= k; j++){
487             if(dp[j-c[i]] != INT_MAX)
```

```
763             dp[j] = min(dp[j], dp[j-c[i]]+1);
273         }
886     }
a5d     if(dp[k] == INT_MAX) cout << -1 << endl;
a71     else cout << dp[k] << endl;
cbb }
```

3.9 Small To Large example

```
66d set<int> distinct[MAXN];
47b vector<int> graph[MAXN];
030 int ans[MAXN];

955 void dfs(int v, int p){
284     for(auto u : graph[v]){
2da         if(u == p) continue;
412         dfs(u, v);
8ec         if(distinct[u].size() > distinct[v].size())
    swap(distinct[u], distinct[v]);
b83         for(auto x : distinct[u]) distinct[v].insert(x);
59c     }
39d     ans[v] = distinct[v].size();
348 }
```

4 Miscellaneous

4.1 Custom Comparator

```
684     int a, b, w;
9ec     bool operator<(const edges &y) { return w < y.w; }
214 };
```

5 String

5.1 ing int64 = long long;

```
4c6 const int64 B = 911382323LL;
9bc const int64 MOD = 1e9+9;
deb const int MAXN = 1e6+6;
dec int64 h[MAXN], pref[MAXN];

0dd signed main(){
9ae     fastio;
bfb     string s, t; cin >> s >> t;

5ce     int n = s.size(), m = t.size();
799     if(m > n){
780         cout << 0 << endl;
bb3         return 0;
726     }
        //preocmputing the powers
e40     pref[0] = 1;
8ee     h[0] = s[0];
78a     for(int i = 1; i <= n; i++){
7de         pref[i] = (pref[i-1]*B)%MOD;
ba3     }

603     for(int i = 0; i < n; i++){
fa8         h[i+1] = (h[i]*B + (s[i]-'a'+1))%MOD;
944     }

6ae     int64 ph = 0;
64a     for(int i = 0; i < m; ++i){
fd8         ph = (ph* B + (t[i] - 'a'+1))%MOD;
658     }

1a4     int ans = 0;
26c     for(int i = 0; i + m <= n; i++){
ecb         int64 hash = (h[i+m] - (h[i]*pref[m])%MOD + MOD)%MOD;
d39         if(hash == ph) ans++;
507     }
886     cout << ans << endl;
e88 }
```

5.2 Manacher

```
//Nao e eficiente pois utiliza substr O(size)

fd3 vector<int> manacher_odd(string s) {
163     int n = s.size();
0cb     s = "$" + s + "^";
1a9     vector<int> p(n + 2);
187     int l = 0, r = 1;
78a     for(int i = 1; i <= n; i++) {
a1d         p[i] = min(r - i, p[l + (r - i)]);
```

```
ce7         while(s[i - p[i]] == s[i + p[i]]) {
fbd             p[i]++;
dc2         }
03e         if(i + p[i] > r) {
26d             l = i - p[i];
a69             r = i + p[i];
4cc         }
283     }
fa1     return vector<int>(begin(p) + 1, end(p) - 1);
08d }

4e5 vector<int> manacher(string s) {
78f     string t;
7e7     for(auto c: s) {
870         t += string("#") + c;
478     }
9cc     auto res = manacher_odd(t + "#");
b50     return res;
cdc }

0dd signed main() {
5d6     winton;
ac0     string s;
a18     cin >> s;

bf9     vector<int> mnc = manacher(s);
35c     vector<pair<int,int>> palindromes;
63e     for (int i = 0; i < mnc.size(); i++){
1e8         if (mnc[i]-1 > 0){
8ad             palindromes.push_back({mnc[i]-1,i});//guarda os
    palindromes com {tamanho, centro}
ad1         }
b1c     }

668     sort(rall(palindromes));//ordena pelos maiores(opicional ne)
603     for (auto &[size, centro] : palindromes){

04c         string pos;
3f0         int st = ((centro - size)/2);
ab5         pos = s.substr(st, size); //gera a substring daquele
    palindrome
2dc     }
bf9 }
```

6 Matematica

6.1 FastExpo

```
5f5 void fastexpo(int a, int b){
c92     int res = 1;
63a     while(b > 0){
615         if(b&1) res = (res*a)%MOD;
49b         a = (a*a)%MOD;
20a         b/=2;
13f     }

478     cout << res << endl;
9ab }
```

6.2 Fatorial e Inverso Modular

```
3e4 const int MOD = 1e9+7;
27e int fat[MAXN], inv[MAXN];

efe void fact(){
557     fat[0] = 1;
fb9     inv[0] = 1;
b74     for(int i = 1; i < MAXN; i++){
d26         fat[i] = (fat[i-1] * i) % MOD;
3ba         inv[i] = fastexpo(fat[i], MOD-2);
1ab     }
31f }

//para calcular o inv[val], basta joga-lo no fastexpo com MOD-2
//para dividir um valor sob mod, basta pegar o multiplicar val *
    inv[val]
```

6.3 GCD

```
4c5 int gcd(int a, int b) { return b == 0 ? a : gcd(b, a % b); }
//lcm(a, b) = a*b/gcd(a, b)
```

6.4 Prime Factorization

```
4ae vector<int> factor(int n) {
4b8     vector<int> ret;
2ed     for (int i = 2; i * i <= n; i++) {
4cd         while (n % i == 0) {
728             ret.push_back(i);
135             n /= i;
59a         }
882     }
44c     if (n > 1) { ret.push_back(n); }
edf     return ret;
9e1 }
```

6.5 Smallest Prime Factor

```
229 const int MAXN = 1e6+7;
bc6 int spf[MAXN];

bea void mfp(){
        //preenche spf[i] com i
9d0     iota(spf, spf+MAXN, 0);
ad0     for(int i = 2; i*i < MAXN; i++){
a58         if(spf[i] == i){
410             for(int j = i*i; j < MAXN; j += i){
480                 if(spf[j] == j) spf[j] = i;
d00             }
b7d         }
eaf     }
d91 }
```

7 Problemas

7.1 Array Division

```
47c int mx;
603 for(int i = 0; i < n; i++){
41e     int a; cin >> a;
2c7     mx = max(mx, a);
e84     arr[i] = a;
e9f }
f6e int l = mx, r = 2e18;
40c while(l < r){
ae0     int mid = (l+r)/2;
7b1     int i = 0, sum = 0, c = 0;
bb5     while(i < n){
c24         if(sum + arr[i] > mid){
eed             sum = 0;
6a4             c++;
88c         }
fd5         sum += arr[i];
0dd         i++;
355     }
79c     if(c < m){
168         r = mid;
e1b     }else{
0dc         l = mid+1;
8cb     }
009 }

46b cout << r << endl;
```

7.2 Distinct Value Queries

```
1a8     int n;
116     vector<int> bit;

5c7     void init(int _n){
81f         n = _n+1;
a72         bit.assign(n, 0);
bc5     }

ff0     void update(int x, int v){
f8f         x++;
aa1         for(; x <= n; x += x&(-x)) bit[x] += v;
774     }

0a4     int sum(int b){
a97         b++;
a93         int sum = 0;
772         for(; b > 0; b -= b&(-b)) sum += bit[b];
e66         return sum;
af6     }
214 };

0dd signed main(){
9ae     fastio;
c5f     int n, q; cin >> n >> q;
04a     int arr[n];
603     for(int i = 0; i < n; i++){
```

```
5d3         cin >> arr[i];
37b     }

e03     BIT bit;
325     bit.init(n);
8c1     map<int, vector<pair<int, int>>> mp;
abf     for(int i = 0; i < q; i++){
b1f         int a, b; cin >> a >> b;
afc         a--, b--;
6f6         mp[a].push_back({b, i});
3fb     }
    //se ja apareceu e ultimo indice
5f3     map<int, int> freq;

913     vector<int> ans(q, -1);
45b     for(int i = n-1; i >= 0; i--){

            //se ele ja foi visto anteriormente, ent retiro a soma dele
d52         if(freq.count(arr[i]) > 0){
626             bit.update(freq[arr[i]], -1);
577         }

1b5         freq[arr[i]] = i;
382         bit.update(i, 1);

            //p/ cada query comecando em i, resposta
e05         for(auto u : mp[i]){
                //cout << i << " " << u.first << " " <<
                    bit.sum(u.first) << endl;
b5c             ans[u.second] = bit.sum(u.first);
def         }
394     }

16b     for(auto u : ans) cout << u << endl;

6e7 }
```

7.3 Divisor Analysis

```
888 int fastexpo(int a, int b){
c92     int res = 1;
630     a%=MOD;
c93     while(b >= 1){
615         if(b&1) res = (res*a)%MOD;
49b         a = (a*a)%MOD;
1b4         b >>= 1;
16b     }
b50     return res;
527 }

0dd signed main(){
9ae     fastio;
9ee     int n; cin >> n;
ef6     vector<pair<int, int>> p;
ad3     int cnt = 1, cnt2 = 1, impar = 1;
603     for(int i = 0; i < n; i++){
f78         int primo, exp; cin >> primo >> exp;
5c9         p.push_back({primo, exp});
30e         cnt = (cnt*(exp+1))%MOD;
ce3         cnt2 = (cnt2*(exp+1))%(2*(MOD-1));
36c         if(exp&1) impar = 0;
116     }

702     int sum = 1, prod = 1;
603     for(int i = 0; i < n; i++){
861         int exp2 = (p[i].second+1)%(MOD-1);

43e         int num = (fastexpo(p[i].first, exp2) - 1 + MOD) % MOD;
e9f         int dem = (p[i].first-1 + MOD)%MOD;
ee9         int inv = (fastexpo(dem, MOD-2));
c43         sum = (sum * ((num * inv)%MOD))%MOD;

4ef         if(!impar){
868             int exp = ((cnt2/2)*p[i].second)%(MOD-1);
d84             prod = (prod * (fastexpo(p[i].first, exp))%MOD)%MOD;
279         }else{
0b9             int exp = ((p[i].second/2)*cnt2)%(MOD-1);
d84             prod = (prod*(fastexpo(p[i].first, exp))%MOD)%MOD;
d08         }
786     }

01b     cout << (cnt%MOD) << " " << (sum%MOD) << " " << (prod%MOD) <<
    endl;
bf5 }
```

7.4 Increasing Subsequence II

```
1a8     int n;
7eb     vector<int> t;
5c7     void init(int _n){
8fd         n = _n;
```

```
dd9         t.assign(4*n, 0);
c5f     }

476     void build(vector<int> &arr, int v, int l, int r){
893         if(l == r){
d60             t[v] = arr[l];
505             return;
303         }
ee4         int m = (l+r)/2;
ebc         build(arr, 2*v+1, l, m);
618         build(arr, 2*v+2, m+1, r);
ab3         t[v] = t[2*v+1] + t[2*v+2];
173     }

6c9     void build(vector<int> &arr){
886         build(arr, 0, 0, n-1);
0dd     }

8fb     int query(int v, int l, int r, int ql, int qr){
dd9         if(qr < l || r < ql) return 0;
639         if(ql <= l && r <= qr) return t[v];
ee4         int m = (l+r)/2;
41a         return query(2*v+1, l, m, ql, qr) + query(2*v+2, m+1, r,
    ql, qr);
2a3     }

9e3     int query(int l, int r){
339         if(l > r) return 0;
5d9         return query(0, 0, n-1, l, r);
fbb     }

b22     void update(int v, int l, int r, int idx, int x){
893         if(l == r){
e60             t[v] += x;
505             return;
4c0         }

ee4         int m = (l+r)/2;
5a5         if(idx <= m) update(2*v+1, l, m, idx, x);
19c         else update(2*v+2, m+1, r, idx, x);
ab3         t[v] = t[2*v+1] + t[2*v+2];
766     }

f18     void update(int idx, int x){
d9b         update(0, 0, n-1, idx, x);
19c     }
214 };
```

```
0dd signed main(){
9ee     int n; cin >> n;
d09     segtree st;
70a     vector<int> v(n);
603     for(int i = 0; i < n; i++){
44f         cin >> v[i];
90c     }

bb2     vector<int> comp = v;
90e     sort(comp.begin(), comp.end());
c91     comp.erase(unique(comp.begin(), comp.end()), comp.end());
    // 2) now comp.size() = M < n
    // 3) to get the index of v[i] use lower_bound(comp, v[i])
294     auto idx = [&](int x){
7d6         return (lower_bound(comp.begin(), comp.end(), x) -
    comp.begin());
158     };

1a4     int ans = 0;
aa8     vector<int> arr(n);
174     st.init(comp.size());
603     for(int i = 0; i < n; i++){
043         int p = idx(v[i]);
cc0         int sum = st.query(0, p-1);
c24         int pd = (1 + sum)%MOD;
fdc         ans = (ans + pd)%MOD;
8ff         st.update(p, pd);
467     }

886     cout << ans << endl;

bce }
```

7.5 Sum of Divisors

```
229 const int MAXN = 1e6+7;
65b const int TWO_MOD_INV = 500000004;

//(b-a+1)*(a+b)2W

030 int sum(int a, int b){
b12     return (((((b-a+1)%MOD) * ((a+b)%MOD))%MOD *
    (TWO_MOD_INV))%MOD);
210 }
```

```
0dd signed main(){
9ae     fastio;
9ee     int n; cin >> n;
1a4     int ans = 0;
65a     int i = 1;
052     while(i <= n){
b19         int quotient = n/i;
58e         int last = n/quotient;

d13         ans = (ans + quotient*(sum(i, last)))%MOD;
6d0         i = last+1;
2a4     }
886     cout << ans << endl;

c1a }
```

8 Extra

8.1 bin.cpp

```
//Estruturas
signed main(){
    int n = 10;
    vector<int> v(n, 1);
    int x = 1;
    auto it = lower_bound(v.begin(), v.end(), x); //msm coisa com
        upperbound
    if(it != v.end()){

    }
}

void sets() {
    set<int> s;
    s.insert(1);                    // [1]
    s.insert(4);                    // [1, 4]
    s.insert(2);                    // [1, 2, 4]
    s.insert(1);                    // does nothing because 1's already
        in the set
    cout << s.count(1) << endl;     // 1
    s.erase(1);                     // [2, 4]
    cout << s.count(5) << endl;     // 0
    s.erase(0);                     // does nothing because 0 wasn't in
        the set
    s.insert(6);                    // [2, 4, 6]

    // Outputs 2, 4, and 6 separated by spaces
    for (int element : s) { cout << element << " "; }

    set<int> s;
    s.insert(1);                            // [1]
    s.insert(14);                           // [1, 14]
    s.insert(9);                            // [1, 9, 14]
    s.insert(2);                            // [1, 2, 9, 14]
    cout << *s.upper_bound(7) << '\n';      // 9
    cout << *s.upper_bound(9) << '\n';      // 14
    cout << *s.lower_bound(5) << '\n';      // 9
    cout << *s.lower_bound(9) << '\n';      // 9
    cout << *s.begin() << '\n';             // 1
    auto it = s.end();
    cout << *(--it) << '\n';        // 14
    s.erase(s.upper_bound(6));      // [1, 2, 14]
```

```
}

void hashsets() {
    unordered_set<int> s;
    s.insert(1);                    // {1}
    s.insert(4);                    // {1, 4}
    s.insert(2);                    // {1, 2, 4}
    s.insert(1);                    // does nothing because 1's already
        in the set
    cout << s.count(1) << endl;     // 1
    s.erase(1);                     // {2, 4}
    cout << s.count(5) << endl;     // 0
    s.erase(0);                     // does nothing because 0 wasn't in
        the set
    s.insert(6);                    // {2, 4, 6}

    // Outputs 6, 2, and 4 separated by spaces (not in sorted order)
    for (int element : s) { cout << element << " "; }
}

void maps() {
    map<int, int> m;
    m[1] = 5;                       // [(1, 5)]
    m[3] = 14;                      // [(1, 5); (3, 14)]
    m[2] = 7;                       // [(1, 5); (2, 7); (3, 14)]
    m[0] = -1;                      // [(0, -1); (1, 5); (2, 7); (3, 14)]
    m.erase(2);                     // [(0, -1); (1, 5); (3, 14)]
    cout << m[1] << endl;           // 5
    cout << m.count(7) << endl;     // 0
    cout << m.count(1) << endl;     // 1
    cout << m[2] << endl;           // 0
    map<int, int> m;
    m[3] = 5;      // [(3, 5)]
    m[11] = 4;     // [(3, 5); (11, 4)]
    m[10] = 491;   // [(3, 5); (10, 491); (11, 4)]
    cout << m.lower_bound(10)->first << " " <<
        m.lower_bound(10)->second << '\n';  // 10 491
    cout << m.upper_bound(10)->first << " " <<
        m.upper_bound(10)->second << '\n';  // 11 4
    m.erase(11);  // [(3, 5); (10, 491)]
    if (m.upper_bound(10) == m.end()) {
        cout << "end" << endl;  // Prints end
    }
}

void multisets(){
    //Using ms.erase(val) erases all instances of val from the
        multiset.
    //To remove one instance of val, use ms.erase(ms.find(val))!
    multiset<int> ms;
    ms.insert(1);                   // [1]
    ms.insert(14);                  // [1, 14]
    ms.insert(9);                   // [1, 9, 14]
    ms.insert(2);                   // [1, 2, 9, 14]
    ms.insert(9);                   // [1, 2, 9, 9, 14]
    ms.insert(9);                   // [1, 2, 9, 9, 9, 14]
    cout << ms.count(4) << '\n';    // 0
    cout << ms.count(9) << '\n';    // 3
    cout << ms.count(14) << '\n';   // 1
    ms.erase(ms.find(9));
    cout << ms.count(9) << '\n';  // 2
    ms.erase(9);
    cout << ms.count(9) << '\n';  // 0
}
```

8.2 template.cpp

```
#include <bits/stdc++.h>
using namespace std;

#define fastio ios_base::sync_with_stdio(0); cin.tie(0)
#define int long long
#define endl '\n'

signed main(){
    fastio;
}
```

8.3 makefile

```
CXX = g++
CXXFLAGS = -fsanitize=address,undefined -fno-omit-frame-pointer -g
    -Wall -Wshadow -std=c++17 -Wno-unused-result -Wno-sign-compare
    -Wno-char-subscripts #-fuse-ld=gold
```